

Inryeol Choi

Backend Portfolio

A project record focused on backend systems that connect service flow, data structure, authentication, and runtime environments.

dlsfuf0316@gmail.com | github.com/InryeolChoi | Seoul, South Korea

Portfolio Focus

- Treat APIs as structured user flows that connect data state, validation, exceptions, and operational behavior.
- Work across foundational backend concerns such as authentication, databases, deployment, and runtime configuration.
- Built system programming fundamentals through hands-on projects involving networking, processes, file descriptors, and concurrency.

Representative Projects

ft_malloc

Thread-safe dynamic memory allocator in C

Implemented malloc, free, realloc, show_alloc_mem, and show_alloc_mem_ex on top of mmap, then extended the allocator for concurrent access.

Problem

A memory allocator must handle alignment, fragmentation, invalid pointers, size overflow, and munmap failure while remaining consistent under concurrent access.

Approach

Tracked allocation with boxes and tag metadata across TINY, SMALL, and LARGE zones. Implemented free-tag reuse, splitting, and coalescing, protected each zone with a pthread mutex, and updated box lists only after munmap succeeds.

Result

Achieved 16-byte alignment and failure-safe release, and verified multithreaded stress runs without deadlocks or memory corruption.

- Implemented malloc, free, realloc, and memory inspection functions.
- Added free-tag lookup, splitting, reuse, and adjacent-tag coalescing.
- Printed live user areas as hexadecimal dumps in show_alloc_mem_ex.

Tech Stack

C	mmap	pthread mutex	Dynamic memory
Thread safety			

Simsimjogak

[Spring Boot backend for an emotion-based diary service](#)

Built a backend that connects social login, diary CRUD, AI letters, summaries, and monthly keyword reports for an emotion-based diary service.

Problem

Social login, token refresh, diary data management, and external AI response generation needed to work together without breaking the user flow.

Approach

Set up authentication around Spring Security and JWT, connected MySQL and Redis to separate user data from token state, and integrated a local AI server. When external AI responses differed from the expected format, I analyzed the response structure and strengthened the parsing logic.

Result

Created a backend structure that handles authentication, persistence, AI response generation, and monthly lookup flow in one service.

- Implemented Google and Apple social login with JWT refresh flow.
- Built diary create, read, update, delete, and monthly record query APIs.
- Analyzed external AI API response structure differences and verified fixes with test cases.

Tech Stack

Spring Boot	Java	Spring Security	JPA
MySQL	Redis	JWT	

Active Recall Quiz

[Study app that turns markdown notes into exam and review flows](#)

A learning app that syncs external markdown notes and turns them into quiz creation, exam submission, grading, and wrong-answer review flows.

Problem

Study materials were scattered across markdown notes, making it difficult to turn them into a repeatable exam, grading, and review cycle.

Approach

Connected the note repository and app through GitHub Actions, then structured the FastAPI backend around exam creation, submission, grading, and result lookup.

Result

Created a flow where note writing and app-based review reinforce each other, including wrong-answer review.

- Structured synchronization for external markdown content.
- Separated exam creation, submission, grading, and result lookup API flows.
- Built the study workflow with a Next.js frontend and FastAPI backend.

Tech Stack

Next.js	React	FastAPI	SQLite
GitHub Actions	Markdown		

ft_irc

Event-driven IRC server

Implemented an IRC server that handles multiple user connections and channel state, learning network server structure and event-driven I/O.

Problem

Multiple client connections, command processing, and channel state could quickly make the server architecture hard to follow.

Approach

Separated the server responsibilities early and centered the implementation around kqueue-based event monitoring for connection and channel state handling.

Result

Built a practical understanding of how event-driven servers process connections, commands, and shared state.

- Designed the server structure before expanding features.
- Studied the event detection flow of kqueue, a multiplexing-based non-blocking I/O system call.
- Learned how a server manages client connections and channel state.

Tech Stack

C++	Socket	kqueue	IRC
Network server			

minishell

Unix shell parsing and process execution

Implemented how a shell interprets input and connects commands, learning parsing and process flow through direct implementation.

Problem

A shell has to structure input according to rules such as pipes, redirection, and environment variables instead of simply executing raw strings.

Approach

Studied how input is parsed into executable structures, then connected commands with pipe, fork, dup2, and execve.

Result

Implemented the full flow from string parsing to execution planning and process chaining.

- Learned the parsing flow that turns input into executable structures.
- Implemented pipe-based command chaining.
- Understood shell command execution and process connection.

Tech Stack

C	Unix process	pipe	fork
dup2	execve		

transcendence

[Service-style web backend with authentication and runtime setup](#)

Built a backend for a real web-service flow, covering authentication, persistence, and runtime environment setup.

Problem

The API server, login, security, database, and deployment runtime needed to work together as one integrated system.

Approach

Built APIs with Django REST Framework and integrated 42 OAuth login, JWT issuance and refresh, and email-based second-factor verification links into the authentication flow.

Result

Configured an authentication flow that considered failures and token expiration, plus a PostgreSQL and Docker Compose runtime environment.

- Built the backend with Django and Django REST Framework.
- Implemented 42 OAuth login, JWT issuance, and email-based second-factor verification links.
- Set up a development environment with PostgreSQL and Docker Compose.

Tech Stack

Django	DRF	PostgreSQL	Docker
JWT	OAuth		

Database Playground / Profile Site / Active Recall Notes

[Supporting repositories for data practice, portfolio publishing, and study content modeling](#)

Maintained supporting repositories for database practice, static site publishing, and markdown-based study content modeling.

Problem

I wanted concept study to remain reusable instead of disappearing after one-time practice.

Approach

Separated a PostgreSQL and Docker SQL practice repository, a React profile site, and a markdown study note repository.

Result

Built a personal development workflow that connects study, documentation, publishing, and automation.

- Practiced databases with PostgreSQL and Docker.
- Built a GitHub Pages profile site with React, TypeScript, and Vite.
- Synchronized markdown notes and learning app content with GitHub Actions.

Tech Stack

PostgreSQL	Docker	React	TypeScript
Vite	Markdown	GitHub Actions	